Worksheet: Designing Pseudocode Subprograms

Scenario: Riverside Community Market (RCM)

Purpose of This Activity

In this activity, you will practise writing **exam-style pseudocode subprograms** based on a realistic scenario.

By the end of the lesson, you should be able to:

- Identify **system processes** from a scenario
- Design **clear pseudocode subprograms**
- Explain **how and why** your pseudocode supports the scenario
- Write answers that gain **maximum exam marks**

What Examiners Are Looking For

In pseudocode questions, examiners reward:

✅ A **clear purpose** for each subprogram
✅ **Sensible inputs and outputs**
✅ Logical use of **IF**, **FOR**, **FUNCTION**, **RETURN**
✅ Strong **links to the scenario**
✅ Evidence of **good system design**, not perfect syntax

They do **not** expect:

- A real programming language
- Error-free code
- The entire system in one answer

Tip: Always explain *why* your pseudocode is useful for the scenario.

Scenario Reminder (Summary)

Riverside Community Market (RCM) is a cooperative of local producers.
The digital system must support:

- Customers browsing products, placing orders, and viewing order history
- Producers managing stock and product information
- Features such as loyalty discounts and secure access

Part 1: Worked Example Subprograms

Study the following **exam-style examples**. You will write your own later. They are incorrectly laid out in some parts, to encourage you to explore the pseudocode worksheet shared by Edexcel.

Subprogram 1: View Available Products

```
SUBPROGRAM ViewAvailableProducts(category)
    FOR EACH product IN ProductList
        IF product.category = category AND product.stock > 0 THEN
            DISPLAY product.name, product.price, product.stock
        END IF
    NEXT product
END SUBPROGRAM
```

Justification

- Supports customers browsing products with transparent pricing and availability
- Uses stock level checks to avoid showing unavailable items
- Links directly to the requirement to show real-time availability

Discussion prompts

- Why is `product.stock > 0` important?
- What problem would occur if this check was missing?
- Who benefits from this subprogram: customer, producer, or both?

Subprogram 2: Place Order

```
SUBPROGRAM PlaceOrder(customerID, productID, quantity)
    IF CheckStock(productID, quantity) = TRUE THEN
        CREATE newOrder
        newOrder.customerID ← customerID
        newOrder.productID ← productID
        newOrder.quantity ← quantity
        UPDATE stock level
        STORE newOrder
        RETURN "Order successful"
    ELSE
        RETURN "Insufficient stock"
    END IF
END SUBPROGRAM
```

Justification

- Supports order placement for collection or delivery
- Prevents orders being placed for unavailable stock
- Demonstrates validation, which examiners like to see
- Links customer actions to producer stock management

Discussion prompts

- Why is validation important in real systems?
- What could go wrong if stock is not checked first?
- Which exam keywords apply here? (e.g. validation, data integrity)

Subprogram 3: Check Stock Levels

```
FUNCTION CheckStock(productID, quantityRequired) RETURNS BOOLEAN
    IF ProductList[productID].stock ≥ quantityRequired THEN
        RETURN TRUE
    ELSE
        RETURN FALSE
    END IF
END FUNCTION
```

Justification

- Modular design (can be reused in multiple places)

- Helps maintain accurate stock data
- Supports both customer ordering and producer dashboards
- Shows good decomposition, a common marking point

Discussion prompts

- Why is this better as a function than repeated code?
- Where else could this function be used?
- How does this improve system reliability?

Subprogram 4: Update Stock (Producer Dashboard)

```
SUBPROGRAM UpdateStock(producerID, productID, newStockLevel)
    IF VerifyProducer(producerID, productID) = TRUE THEN
        ProductList[productID].stock ← newStockLevel
        RETURN "Stock updated"
    ELSE
        RETURN "Access denied"
    END IF
END SUBPROGRAM
```

Justification

- Directly supports the producer portal
- Demonstrates access control
- Shows awareness of security and data ownership
- Links to market research requirements

Discussion prompts

- Why should customers not be allowed to run this?
- What real-world risk does `VerifyProducer` reduce?
- Which non-functional requirement does this support?

Subprogram 5: Apply Loyalty Discount

```
FUNCTION ApplyLoyaltyDiscount(customerID, orderTotal) RETURNS REAL
    IF Customer[customerID].loyaltyPoints ≥ 100 THEN
        discount ← orderTotal * 0.1
        RETURN orderTotal - discount
    ELSE
```

```
        RETURN orderTotal
    END IF
END FUNCTION
```

Justification

- Supports the loyalty scheme feature
- Uses stored customer data
- Demonstrates conditional logic with real-world impact
- Easy marks for linking logic to business needs

Discussion prompts

- Why use a function rather than a subprogram?
- How could the discount rule be changed easily?
- What happens if loyalty points are stored incorrectly?

Subprogram 6: View Order History

```
SUBPROGRAM ViewOrderHistory(customerID)
    FOR EACH order IN OrderList
        IF order.customerID = customerID THEN
            DISPLAY order.details, order.date, order.status
        END IF
    NEXT order
END SUBPROGRAM
```

Justification

- Meets the requirement for customers to view order history
- Supports account management features
- Reinforces trust and transparency

Discussion prompts

- Why is filtering by `customerID` essential?
- What data protection issue could arise here?
- How does this improve user experience?

Part 2: Your Turn (Exam Practice)

Task A: Identify Subprograms (10 minutes)

From the scenario, list **three additional system actions** that could be written as subprograms.

Example sentence starter:

"The system needs a subprogram to…"

Task B: Write Pseudocode (20 minutes)

Choose **two** of your actions and:

- Write an exam-style subprogram or function
- Use IF, FOR, or RETURN where appropriate
- Focus on **clarity**, not perfection

Task C: Justify Your Design (10 minutes)

For each subprogram, write **2–3 sentences** explaining:

- What requirement it supports
- Why it improves the system
- Which users benefit

Final Exam Reflection (Class Discussion)

- Which subprogram would score the most marks?
- Where did validation or security appear?
- How could you improve your answer under time pressure?

Exam Reminder:
Pseudocode + clear justification = higher marks.